

DartBud

Gruppe 1

Jonas El Helling – Kandidat: 7065

Kristian Martin Tvenning – Kandidat: 7124

Kenneth Andreas Hansen – Kandidat: 7111

Lucas Leon Svaet Holter – Kandidat: 7107

M1. Contribution Claim (Concept, Programming, design, Report)

	Concept	Programming	Design	Report	Overall Contribution* *
Jonas	20%	20%	20%	30%	25%
Kristian	20%	30%	30%	25%	25%
Kenneth	40%	30%	30%	25%	25%
Lucas	20%	20%	20%	20%	25%

M2. Chosen app features

Features	Reference (in Code/Activity/Module)	Complete/Not?
Feature 1	Kalkulering av dobbel/trippel poeng	✓
Feature 2	Google Sign-In Authentication with Firebase	✓
Feature 3	Local Database with Room	✓
Feature 4	Real-time spill statistikk (average, runder spilt, piler kastet)	✓
Feature 5	Avansert Undo system (Undo helt tilbake til første pil)	✓
Feature 6	Checkout forslag (Hva man må kaste for å vinne spillet)	✓
Feature 7	Oppdager Bust ved ugyldig score	✓
Feature 8	Double in/Double out (Avanserte regler)	✓
Feature 9	Responsive UI for telefoner med forskjellige størrelser på skjerm	✓
Feature 10	Game History Gjennomførte spill blir lagret med statistikk.	✓
Feature 11	Player Management Lage, slette, endre på spillerprofiler lokalt eller innlogget Googlekonto.	✓
Feature 12	Rematch - mulighet for rematch med knapp	✓
Feature 13	Exit bekreftelse hvis man forlater midt i et spill	✓
Feature 14	Darkmode	✓
Feature 15	Rules-screen	✓
Feature 16	Turn-indicator system Rosa outline rundt gjeldende spiller	✓
Feature 17	LED-style score display	✓

M3. App Running essentials:

- Android phone with screen and battery.
- Allowing apps from external sources (if downloaded .apk from GitHub)

Innhold

DartBud	1
M1. Contribution Claim (Concept, Programming, design, Report)	2
M2. Chosen app features	2
M3. App Running essentials:	3
Introduction.....	4
Problem Space.....	5
3. Project Organization.....	7
Teammedlemmer og roller.....	7
Hvorfor valgte dere dette prosjektet?.....	7
Utviklingsmetode.....	7
Requirements Analysis and Design.....	7
Funksjonelle krav.....	7
Ikke-funksjonelle krav.....	14
Systemarkitektur (DB, UI, server, client etc. Components).....	15
Informasjonsarkitektur/flytdiagram.....	15
Flytdiagram.....	15
Working Process.....	16
Samarbeidsprosess og verktøy (Git, Trello etc.).....	16
Technical Description.....	18
Beskrivelse av valgte features (database, navigasjon, valgfrie funksjoner).....	18
Error handling:.....	19
Komponenter og biblioteker brukt.....	19
Biblioteker:.....	19
End Product.....	24
Installering og kjøreinstruksjoner.....	24
App-funksjonalitet.....	26
Brukermanual (valgfritt/kort).....	28
Testing.....	28

Evaluation Conclusion.....	29
Hva er ferdig/ikke ferdig.....	29
Fremtidige forbedringer.....	29
Individuell refleksjon.....	30

Introduksjon

DartBud skal hjelpe nybegynnere, amatører og hobbyspillere å holde orden på hoderegningen i den populære dartspill-varianten 501. Tanken er at man setter opp hvem som skal spille, trykke start og så er man i gang med telefonen på bordet som holder tellingen på total score og hjelper deg underveis med hoderegningen.

Appen er altså et hjelpemiddel til den fysiske dartspillingen, og er en slags fokusert kalkulator. I tillegg til å holde rede på utregningen, holder den også rede på hvilket pilkast-nummer det er, og hvem sin tur det er.

Det finnes alternativer til dette, for eksempel MyDartTraining som er den appen som brukes blant vår målgruppe i praksis. Men, her er det en del hoderegning som må til, og som lett kan klusse opp og sette en stopper for en rekreasjonell kveld. Pluss at den appen har ekstremt mange funksjoner, som igjen blir uoversiktlig.

Vårt alternativ eliminerer hoderegningen med introduksjonen av «dobbel» og «trippel»-knappene. Man kaster en pil (throw), og logger dermed scoren, f.eks dobbel 15, og trykker da på «1» + «5» + «2x» og bekrefter denne scoren ved å trykke på checkmark-knappen. Da vil det automatisk gå videre til throw 2 og man repeterer prosedyren.

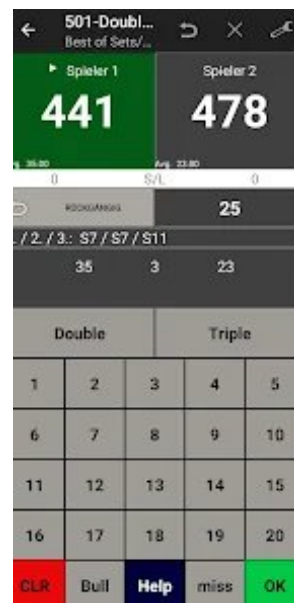
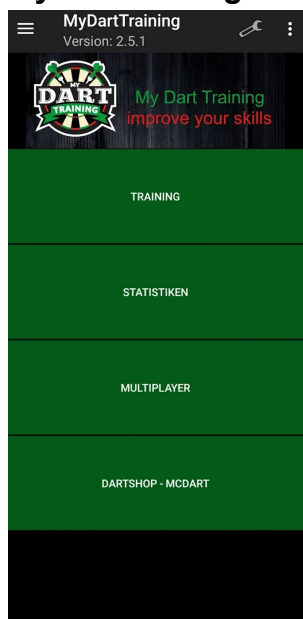
Dette er ikke tilgjengelig i MyDartTraining. Der må man manuelt regne ut scoren for alle tre kastene for å legge inn. Hvis man får trippel 17, trippel 19, trippel 18 på sine tre kast blir det fort krøll i hoderegningen. I tillegg er GUI-et i MyDartTraining utdatert, og knappene varierer fra å være kjempesmå til store. Vår ser mer moderne ut, og er i større grad designet for menneskefingre, også mannlige pølsefingre.

Problemet

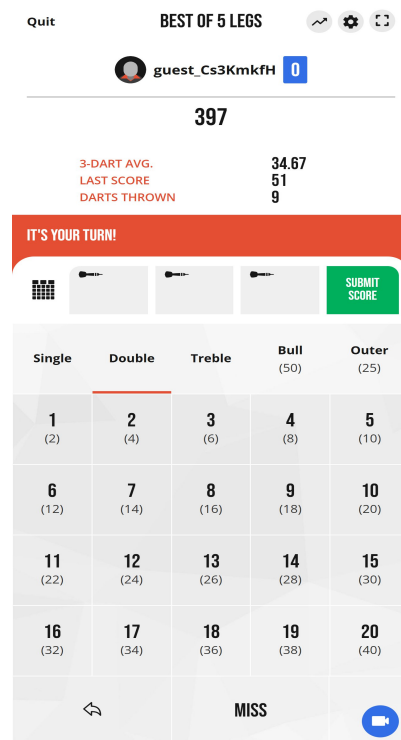
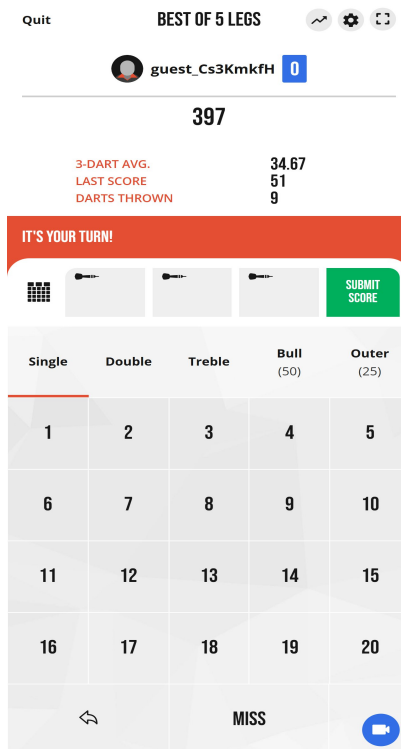
Problemet med MyDartTraining er utregningen. Det blir fort kluss med hoderegning, spesielt de vanskeligere utregningene, og dette blir ikke bedre om man i tillegg drikker øl samtidig. Vår app legger seg helt opp til hva man faktisk sier i praksis at man treffer. Får man en «trippel 18», så trykker man på «3x» + «1» + «8», og appen regner ut resultatet. Siden faktorenes orden er likegyldig, er også input'en det. Det betyr at man også kan skrive inn «1» + «8» + «3x» om man ikke tenker «trippel atten», men «atten ganger tre». Slik sikrer man at det ikke blir knuffing om hoderegning osv. Man vil gjerne unngå å sitte med en kalkulator for å holde flyten i spillet, litt som hooligans som er imot VAR. MyDartTraining er også uoversiktlig med

et rotete brukergrensesnitt, og det er mange funksjoner og valgmuligheter som simpelthen blir for mye å forholde seg til for hobbyspillere. Lignende apper er forenevnte MyDartTraining, DartCounter, Dart Teller: Scores Kalkulator osv. Vår er annerledes ved å være spisset inn på typen Dart 501, den vanligste, og er mer fokusert på hobbyspillere med akkurat den informasjonen man trenger, og er ikke «overlesset» med valgmuligheter eller info man ikke er interessert i. Derfor har vi også et bedre utgangspunkt for et mer intuitivt og minimalt brukergrensesnitt, som kan ta oppmerksomheten fra telefonskjermen til dartbrettet.

MyDartTraining



MyDartTraining er en litt for overkomplisert app med utdatert utseende (UI/UX). Knappene er også små, i hvert fall om man ikke har en mobil med stor skjerm. Trekker også ned at det er mye reklame/popups i appen.



Dartcounter er den mest populære appen på markedet. Den oppleves ikke som veldig intuitiv, og fremstår som rotete og vanskelig å navigere i. Denne koster også penger for å slippe reklame, og brukergrensesnittet fremstår som litt for mye å navigere rundt, lite intuitivt.

3. Project Organization

Teammedlemmer og roller

- Kenneth Andreas Hansen, software architect
- Jonas El H'babi Helling, project manager
- Lucas Leon Svæet Holter, database
- Kristian Martin Tvenning, UX/UI

Hvorfor valgte dere dette prosjektet?

Kenneth er medlem av dartklubben og har sett et behov innenfor denne subkulturen i Hønefoss, hvor man spiller rekreasjonelt. Da drikker man gjerne øl samtidig, og som følge av det går man fort på en snurr i utregninger og da kan det oppstå håndgemeng. Vår app motvirker dette ved å ha et brukervennlig UI/UX design og ved å outsource de matematiske beregningene til en datamaskin.

Utviklingsmetode

Det er i praksis agile development. I flow state-tider er det extreme programming, med mange commits og pushes til GitHub-repoen. Vi *har* også en overordnet plan

(Gantt-chart), men denne havner litt bakpå etter at vi ser større behov i praksis. Den er heller ikke så detaljert at den er noe særlig brukbar når vi er «midt i smørøyet».

Requirements Analysis and Design

Funksjonelle krav

- Systemet skal la brukeren registrere 2 spillere for en runde.
- Systemet skal støtte registrering av konto med e-post og Google SSO.
- Systemet skal la brukeren gi navn til spillere (f.eks. endre “Spiller 1” til ønsket navn).
- Systemet skal la brukeren registrere kast.
- Systemet skal vise hvem sin tur det er til å kaste.
- Systemet skal lagre spill-statistikk.
- Systemet skal tilby multiplikator-knapper for doble og triple treff.
- Systemet skal la brukeren administrere brukerdata (Gjøres fra Googles sider for Google-innlogging, eller «Tøm mellomlager» i Android for gjestemodus).
- Systemet skal vise score på forrige kast.
- Systemet skal vise score.
- Systemet skal la brukeren velge dobbel checkout.
- Systemet skal tilby en Activity-view med regler.
- Systemet skal la brukeren velge dobbel checkout-regel.
- Systemet skal gi checkout-forslag.
- Systemet skal la brukeren sette dobbel check-in.

Beskrevet i nærmere detalj i use-cases:

Use case navn: «Det må være mulig å registrere konto med e-post/Google SSO»	
Rolle: Person som vil bli app-bruker	
Beskrivelse: For å kunne lagre statistikk i lengre tid og ikke lokalt på telefon (og evt dele og sammenligne statistikk) lagrer man en konto hvor denne infoen er bundet til epost	
Trigger:	
Hovedflyt: • GUI viser registreringsskjema • Bruker fyller inn e-postadresse og passord • System validerer e-postformat og passordstyrke • System sjekker at e-post ikke allerede er registrert • System sender bekreftelse til e-post • Bruker bekrefter via e-post	Alternativ flyt • 3a. Ugyldig e-postformat → System viser feilmelding • 4a. E-post allerede registrert → System foreslår innlogging • 6a. Bekreftelse ikke mottatt → System tilbyr å sende på nytt

Konto aktiveres	

Use case navn: «Det må være mulig å gi navn til {spiller 1} slik at vedkommende kan hete Kåre eller Kari i stedet	
Rolle: App-bruker	
Beskrivelse: I feltet for {Spiller 1}+1 oppgir man ønsket navn i et Text Input-field	
Trigger: App-bruker vil angi hvor mange spillere som skal spille	
Hovedflyt: - GUI viser navnefelt for aktuell spiller (f.eks. "Spiller 1") - App-bruker skriver inn ønsket spillernavn - System validerer at navnet ikke er tomt - System lagrer spillernavnet - GUI går videre til neste spiller (hvis flere) eller bekrefter alle navn	Alternativ flyt 2a. Bruker lar feltet stå tomt → System bruker standardnavn "Spiller X" 3a. Navn er for langt → System viser feilmelding og begrenser antall tegn 2b. Bruker velger å bruke eksisterende spiller → System henter navn fra database

Use case navn: «Registrere kast»	
Rolle: App-bruker	
Beskrivelse: Under spillet registrerer bruker hvert kast for å holde rede på poeng og beregne gjenværende score	
Trigger: Spiller har kastet dart og vil registrere resultatet	
Hovedflyt: - GUI viser kastregistrering for aktiv spiller - App-bruker velger dart-verdi (1-20, bull, miss) - App-bruker velger multiplikator (enkelt, dobbelt, trippel) - System beregner kastverdi og legger til totalsum for runden - System oppdaterer gjenværende poeng (501 - totalkast) - GUI viser oppdatert score	Alternativ flyt 2a. Bruker registrerer ugyldig kombinasjon → System viser feilmelding 5a. Kast resulterer i "bust" (under 0 eller 1 ved dobbel checkout) → System nullstiller runden 3a. Bruker angrer kast → System tillater redigering av siste kast

Use case navn: «GUI må vise hvem sin tur det er til å kaste»	
Rolle: Aktiv spiller	

Beskrivelse: GUI highlighter hvem det er sin tur til å kaste med en pil/outline	
Trigger: Ved innsettelse av score (enter), går turen automatisk til neste spiller	
Hovedflyt: Rosa outline rundt Player-card, rosa outline rundt spillers kast	Alternativ flyt

Use case navn: Lagre spillstatistikk	
Rolle: App-bruker	
Beskrivelse: System registrerer og lagrer statistikk for hvert kast og spill for senere analyse	
Trigger: Spiller registrerer kast eller fullfører spill	
Hovedflyt: - System registrerer hvert kast med verdi og multiplikator - System beregner gjennomsnittskast per spiller - System teller spesielle kast (180, 140+, bull, etc.) - System registrerer checkout-forsøk og suksessrate - System lagrer data i database koblet til spillerprofil	Alternativ flyt 5a. Database utilgjengelig → System lagrer lokalt midlertidig 1a. Ugyldig kast registrert → System korrigerer statistikk når kast redigeres

Use case navn: «Det må være mulig å kunne bruke multiplikator-knapper på doble og triple treff»	
Rolle:	
Beskrivelse: Når bruker registrerer kast, må de kunne velge om kastet traff enkelt, dobbelt eller trippel område på dartsbrettet	
Trigger:	
Hovedflyt: - GUI viser kalkulator-interface med sifferknapper (0-9) og Bull-knapp - App-bruker bygger tallet ved å trykke siffer (f.eks. 2, 0 for "20") ELLER trykker Bull - GUI viser multiplikator-knapper (1x, 2x, 3x) eller (S, D, T) - Hvis Bull er valgt: System grår ut 3x/T-knappen (kun enkelt og dobbel tilgjengelig) - App-bruker trykker på tilgjengelig multiplikator	Alternativ flyt - Bruker taster inn ugyldig dartverdi (>20) → System viser feilmelding - Bruker har tastet tall >20 og velger 3x → System viser feilmelding (kun 1-20 har trippel) - Bull × 1 = 25 poeng, Bull × 2 = 50 poeng (bull's eye) - Bruker trykker "clear/C" → System nullstiller inntasting og aktiverer alle multiplikatorer igjen - Bruker taster "0" → System registrerer som miss (0 poeng)

• System beregner kastverdi (inntastet tall × multiplikator eller Bull × multiplikator) • System viser beregnet kastverdi og legger til total • App-bruker bekrefter kastet eller kan redigere før bekreftelse	

Use case navn: Administrere brukerdata	
Rolle: App-bruker	
Beskrivelse: Bruker kan redigere spillerinformasjon eller slette spillerprofiler	
Trigger: Bruker ønsker å oppdatere eller fjerne spillerdata	
Hovedflyt: • GUI viser liste over registrerte spillere • App-bruker velger spiller å redigere eller slette • For redigering: GUI viser redigerbart skjema med eksisterende data • App-bruker gjør endringer og bekrefter • System validerer og lagrer oppdaterte data	Alternativ flyt • Bruker velger sletting → System spør om bekreftelse før sletting • Spiller har aktive spill → System advarer om konsekvenser • Validering feiler → System viser feilmelding og lar bruker korrigere

Use case navn: «Vise score»	
Rolle: app-bruker	
Beskrivelse: Appen skal vise hva man har i score i game-skjerm	
Trigger:	
Hovedflyt: Scoren vises i Player Card med stor skrift	Alternativ flyt

Use case navn: «Det må være mulig velge dobbel checkout (har man 16, må man få dobbel 8 for å vinne)»	
Rolle: App-bruker	

Beskrivelse: Ved opprettelse av spill er det en Checkbox for «Dobbel checkout». Denne må hukes av for å sette at man har med dobbel checkout, ellers er det ikke nødvendig.	
Trigger:	
Hovedflyt: 1. Trykk på Switchen for å slå funksjon på eller av	Alternativ flyt

Use case navn: «Det må være en Activity-view hvor reglene står»	
Rolle: App-bruker	
Beskrivelse: Dart 501 er Dart-varianten i vår MVP. I denne varianten er det en del regler, og en del variasjoner. Disse reglene står i dette viewet.	
Trigger: Man ønsker å sette seg inn i reglene	
Hovedflyt: Entres via hovedmeny	Alternativ flyt

Use case navn: « Det må være mulig å velge dobbel checkout-regel »	
Rolle: app bruker	
Beskrivelse: Bruker kan velge om spillet skal følge regelen om at man må avslutte med dobbel	
Trigger: Bruker ønsker å konfigurere spillregler før spillstart	
Hovedflyt: • GUI viser spillinnstillinger • App-bruker ser avkrysningsboks for "Dobbel checkout" • App-bruker velger å aktivere eller deaktivere funksjonen • System lagrer innstillingen for det aktuelle spillet • System tilpasser checkout-beregninger basert på valget	Alternativ flyt • 3a. Bruker endrer innstilling midt i spill → System spør om bekreftelse • 5a. Dobbel checkout aktivert og spiller står på 1 poeng → System viser "bust"

Use case navn: « Det må være mulig å få checkout-forslag »	
Rolle: app bruker	
Beskrivelse: System foreslår mulige kombinasjoner for å avslutte spillet basert på gjenværende poeng	
Trigger: Spiller har 170 eller færre poeng igjen	
Hovedflyt: • System detekterer at spiller har checkout-mulighet • System beregner mulige kombinasjoner basert på gjenværende poeng	Alternativ flyt • 2a. Ingen mulige checkout-kombinasjoner → System viser melding om å redusere score først • 4a. Dobbel checkout påkrevd

• GUI viser liste over checkout-alternativer • System tar hensyn til dobbel checkout-regel hvis aktivert • App-bruker kan se forslagene som veiledning	men ingen dobbel-finish mulig → System viser kun alternative reduksjoner

Rolle: App-bruker	
Beskrivelse: Det må være mulig å åpne "innstillinger" for appen	
Trigger: App-bruker vil gå på innstillinger fanen	
Hovedflyt: 1. GUI viser "flere alternativer" knapp 2. GUI viser liste med flere alternativer hvor innstillinger er ett alternativ. 3. Bruker trykker på innstillinger knapp 4. GUI viser liste med innstillinger	Alternativ flyt

Use case navn: «Sette dobbel check in»	
Rolle: app bruker	
Beskrivelse: Valg av dobbel in for å starte spillet - spilleren må treffe dobbel for å begynne å telle poeng	
Trigger: App-bruker ønsker å konfigurere spillregler før spillstart	
Hovedflyt: • GUI viser spillinnstillinger/regelvalg • App-bruker ser avkrysningsboks for "Dobbel in" • App-bruker velger å aktivere eller deaktivere dobbel in-regel • System lagrer innstillingen for det aktuelle spillet • Hvis aktivert: System krever dobbel på første scoring-kast for hver spiller • System viser regelstatus i spillgrensesnitt	Alternativ flyt • Bruker endrer innstilling midt i spill → System spør om bekreftelse og restart • Spiller kaster ikke dobbel på første forsøk → Kastet teller ikke, poeng forblir 501 • Spiller treffer dobbel → System starter normal poengtelling fra det kastet

Ikke-funksjonelle krav

- Viktige handlinger skal være tilgjengelige med maks 2–3 trykk.
- UI skal være rent og lett forståelig.
- Sync med backend skal tåle midlertidig nettutfall og automatisk forsøke på nytt (retry).
- Om sync med backend får timeout, skal dataen lagres midlertidig på enheten til neste nett-tilkobling, for så å laste den opp da.
- Skal ikke be om mer data enn nødvendig, det er ikke behov for å vite DOB, lokasjon, tilgang til galleri osv, men krever innlogging/skrivetilgang (for lokal lagring).
- Privacy og dataminimering - Guest-brukere skal kun få lagret minimal nødvendig data (vinner, regler, tidspunkt) - Full statistikk (average, darts thrown, etc.) skal kun lagres for innloggede brukere - Dette følger GDPR-prinsippet om dataminimering (Datatilsynet, 2019).

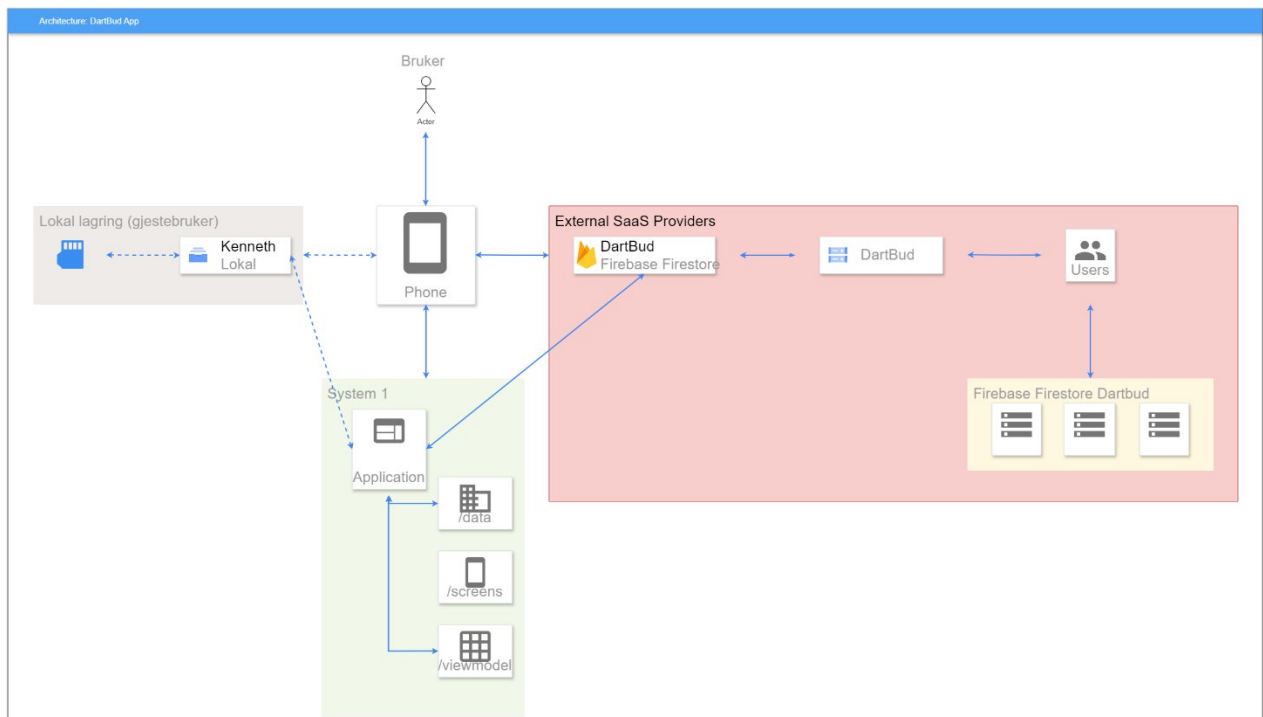
DRØMMESCENARIO TIL FREMTID:

1. Integrere med kamera (Spiller a og b kan filme sine dart brett med mikrofon, online og kan snakke sammen)
2. Valg mellom flere "game-modes" som 101, 201... Mickey Mouse osv.
3. Kan trykke på virtuelt dartbrett for å velge poeng. (mulig kun for pad/tablett versjon)
4. Kan bruke kamera for å filme dartbrett, bruke OpenCV til å gjenkjenne spillers piler, og plassering på brettet for automatisk input av scores
5. Direct Injection for å minimere kodebase og boilerplate-kode
6. Spiller 1 og Spiller 2 skal kunne lagre sine spill på hver sin Google-konto.

Systemarkitektur (DB, UI, server, client etc. Components)

I prosjektet vårt benytter vi Kotlin til business logic og UI, med en lokal SQLite-database til å koble oss opp på den mobile enheten og lokal lagring om man spiller lokalt. For autentisering bruker vi Googles innebygde påloggingssystem. I tillegg bruker vi Firebase som backend/cloud-tjenester (for eksempel til databaser, autentisering og/eller serverløs logikk, avhengig av behovet vi lander på).

Informasjonsarkitektur/flytdiagram



Flytdiagram

Bruker får tilgang til systemet gjennom sin mobil-enhet. På den åpner man tilgang for System 1 og System 2 (Lokal lagring). System 1 ringer hjem til Googles Firebase Firestore og aksesserer DartBud-databasen hvor brukere og deres spill-data lagres. Velger man å ikke logge seg inn med Google kan man spille lokalt. Da lagres spilldataen lokalt på telefonen. Dette kan være nyttig i tilfeller der man ikke har internettdekning.

Working Process

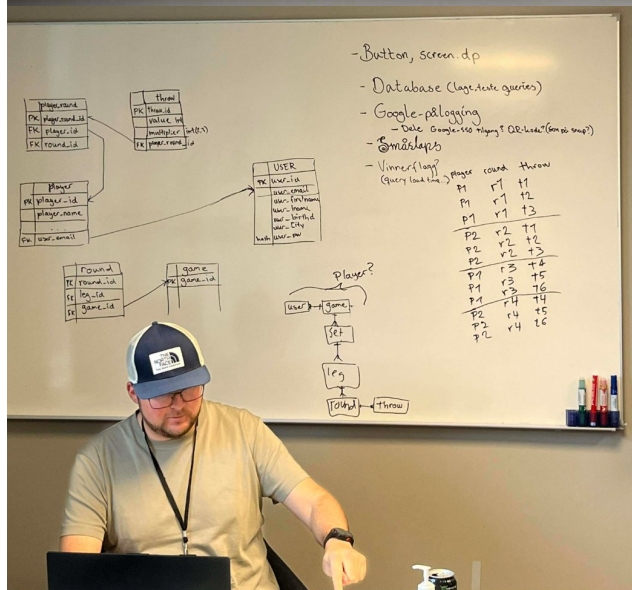
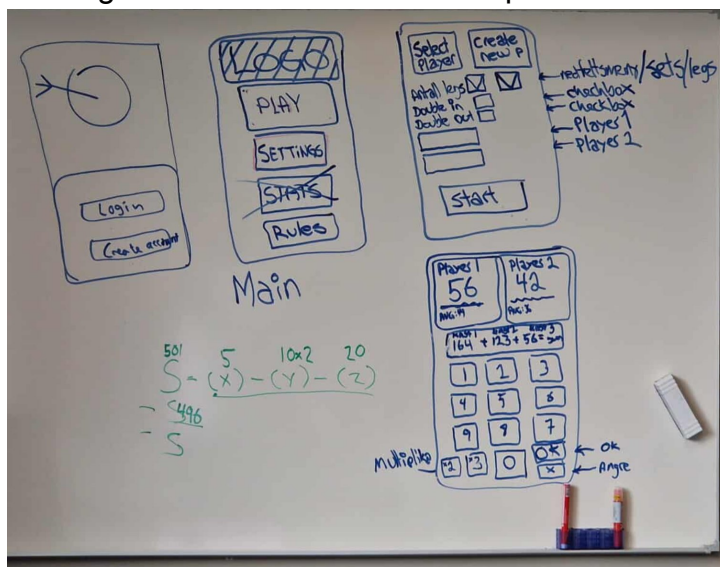
Samarbeidsprosess og verktøy (Git, Trello etc.)

Vi bruker GitHub for å sentralisere repositoryet. Git for versjonskontroll, og logge fremgangen med commit messages. Messenger for å organisere møtetider og holde rede på hvilke rom vi befinner oss på. Vi holder møtene face to face IRL og har fokus på utvikling hele tiden, i stedet for «å vente på hverandre» asynkront som Kristian hører er et problem på andre grupper som hovedsakelig jobber online. Dette konkretiserer fremgangen i prosessen en del, i stedet for at «vi burde ha...» blir hengende i planleggings-fasen uten at man gjør noe med det i det lengste, og så kjører skippertak i slutten. Det hjelper også med å holde fokus, unngå scope creep,

og det at man ender opp med beslutningstrøtthet (decision fatigue), i tillegg til at det gir en god work-life balance. Vi skriver opp oppgaver på tavla under møtene. Vi prøver også å avslutte hvert møte med å lage en oversikt over hva vi skal gå igjennom ved neste arbeidsøkt slik at man kan starte å jobbe umiddelbart.

Oppgavefordeling

Vi har jobbet litt «her og der» med oppgaver. De har blitt kommunisert muntlig og gjort med en gang, de har blitt skrevet opp på tavle og blitt glemt, skrevet opp og merket med haketegn, og skrevet opp og gjort. Det har ikke blitt gjort så mye formell delegering da vi har jobbet sammen i flere emner/prosjekter allerede og har god erfaring med at vi får fordelt arbeid på en effektiv måte.



Timeplan og planlegging

Ved oppstart av prosjektet så lagde vi et ca. utkast av hvordan vi så for oss at vi skulle benytte semesteret og ukene til å arbeide med prosjektet. Det vi kom frem til var at vi ønsker å bruke de to første ukene til planlegging av prosjektet, hvordan vi ønsket å jobbe med prosjektet, når vi skulle jobbe osv. De resterende ukene skulle brukes til utvikling og de tre sistene ukene skulle forhåpentligvis brukes til testing og ferdigstilling avhengig av den endelige eksamensdatoen.

Ukesinndeling														
UKE --->	35	36	37	38	39	40	41	42	43	44	45	46	47	48
	Planlegging		Utvikling									Testing & ferdigstilling		

Eksterne faktorer														
UKE -->	35	36	37	38	39	40	41	42	43	44	45	46	47	48
	Uke 36: Arbeidskrav A1		Uke 39: Arbeidskrav A2				Uke 41: Presentasjon av arbeidskrav					Uke 47: Eksamen innlevering		
			Uke 40: Høstferie (ingen jobbing)				Uke 43: A4 presentasjon							

Ukesplan													
UKE --->	36	37	38	39	40	41	42	43	44	45	46	47	48
Kenneth	F	F	F	F	F	HØSTFERIE	F, M	F, M	T	T	T	M, T, O, T, F	T
Jonas	F	F	F	F	F	HØSTFERIE	F, M, L	F, M	T	T	T	M, T, O, T, F	T
Kristian	F	F	F	F	F	HØSTFERIE	F, M, L	F, M	T	T	T	M, T, O, T, F	T
Lucas	F	F	F	F	F	HØSTFERIE	F	F	T	T	T	M, T, O, T, F	T
Gruppemedlemer													

Plan:
Arbeider fast hver fredag, mer ved behov og kapasitet.
Dette ble endret til tirsdager fast da dette passet gruppen bedre
Klokkeslett: 09:30 - 15:00

Technical Description

Beskrivelse av valgte features

I DartBud har vi implementert følgende programmeringsmessige features:

Jetpack Compose (UI Framework)

Hva: Et moderne deklarativt UI-rammeverk for Android

Hvor brukt: Alle screens (GameScreen.kt, LoginScreen.kt, MainMenuScreen.kt, GameHistoryScreen.kt, RulesScreen.kt), Ctrl+F etter @Composable

Hvorfor: Mindre kode enn XML, bedre state management, Googles anbefalte løsning

Eksempel:

```
@Composable
fun GameScreen() {
    Column {
        PlayerCard(player1, isActive = currentPlayer == 1)
        ThrowButtons(throw1, throw2, throw3)
        NumberPad(onNumberClick = { ... })
    }
}
```

Navigation Component

Hva: Jetpack Navigation for skjermnavigering (Tilbakeknapp gjør at gjeldende skjerm fjernes fra stabelen automatisk gjennom å kalle navController.popBackStack() slik at man sligger å manuelt administrere hvilken skjerm som er aktiv

Hvor brukt: MainActivity.kt med routes:

- "login" → LoginScreen
- "game_screen/{doubleIn}/{doubleOut}/{player1}/{player2}" → GameScreen
- "main_menu" → MainMenuScreen
- "game_history" → GameHistoryScreen

Hvorfor: Håndterer back stack automatisk, type-safe argumenter, færre runtime-feil, bedre IDE-autofullføring, lettere refaktoring, selvdokumenterende kode

Room Persistence Library (Local Database)

Android Jetpack sitt ORM for lokal SQLite database. Konverterer mellom objektorientert kode og relasjonsdatabaser.

Hvor brukt:

Data Layer (com.group1.dartbud.data/):

- PlayerEntity.kt - Spillerprofiler (playerId, username, email, createdAt)

- GameEntity.kt - Fullførte kamper (gameId, player1Id, player2Id, winnerId, doubleIn/Out)
 - GameStatsEntity.kt - Statistikk (average, highestScore, dartsThrown, roundsPlayed)
 - PlayerDao.kt / GameDao.kt - CRUD operasjoner
 - AppDatabase.kt - Database container
 - PlayerRepository.kt / GameRepository.kt - Abstraksjonslag til ViewModels
- Hvorfor: Lokal lagring som fungerer offline (kritisk for dartspilling, kjeller uten mobildekning, etc.), observable data via Flow, ikke manuell SQL-skriving, Room håndterer data-konvertering automatisk (type-safe queries), innebygd error handling.

Eksempel:

```
@Entity(tableName = "players")
data class PlayerEntity(
    @PrimaryKey(autoGenerate = true) val playerId: Int,
    val username: String,
    val email: String?,
    val createdAt: Long
)
```

```
@Dao
interface PlayerDao {
    @Insert
    suspend fun insertPlayer(player: PlayerEntity): Long

    @Query("SELECT * FROM players")
    fun getAllPlayers(): Flow<List<PlayerEntity>>
}
```

Firebase Authentication (Google Sign-In)

Firebase service for brukerpålogging med Google

Hvor brukt:

AuthViewModel.kt - Håndterer authentication state

LoginScreen.kt - Google Sign-In UI

Sikker autentisering, enkel brukeropplevelse, fremtidig cloud sync.

MVVM Architecture Pattern

Model-View-ViewModel arkitekturmønster

Struktur:

Model (Data): PlayerEntity, GameEntity, DAOs, Database

ViewModel (Logic): PlayerViewModel, GameViewModel, AuthViewModel

View (UI): GameScreen, LoginScreen, MainMenuScreen

Data flow: UI → ViewModel → Repository → DAO → Database

Hvorfor: Separation of concerns, enklere testing, anbefalt av Google

Kotlin Coroutines & Flow

Asynkron programmering for å unngå å blokkere UI-tråden

Hvor brukt:

ViewModels for database-kall: `viewModelScope.launch { ... }`

Room DAOs returnerer `Flow<List<...>>` for observable data

UI observerer Flow med `collectAsState()`

Hvorfor: Enklere enn callbacks, integrerer perfekt med Room og Compose

Material Design

Googles design system med ferdiglagde komponenter.

Blir brukt i Button, OutlinedButton, Card, AlertDialog, CircularProgressIndicator

Custom theme i `ui/theme/Theme.kt` med DartBud farger.

Dark mode support.

Dette for å få konsistent design, accessibility innebygd samt et moderne utseende på appen.

Error Handling (Network Utilities)

Custom NetworkUtils klasse for nettverkssjekk

Blitt brukt iblant annet:

- NetworkUtils.kt (utils/) - Sjekker internett-tilkobling
- LoginScreen.kt - Blokkerer Google Sign-In uten nett
- Error dialogs (Network, Bust, Exit, Winner)

Hvorfor: Oppfyller kurskrav, forhindrer crashes, bedre UX

Eksempel:

```
object NetworkUtils {  
    fun isNetworkAvailable(context: Context): Boolean {
```

```

        val connectivityManager = context.getSystemService(...)
        return capabilities.hasCapability(NET_CAPABILITY_INTERNET)
    }
}

```

```

// I LoginScreen:
if (!NetworkUtils.isNetworkAvailable(context)) {
    showNetworkError = true
} else {
    launcher.launch(googleSignInIntent)
}

```

State Management (Compose State)

Hva: Dette er Compose sitt innebygde state system med `remember` og `mutableStateOf`

Blir brukt i GameScreen.kt for player state, throw state, input state, dialog state

Hvorfor: Dette for en automatisk UI recomposition og «enkel» syntaks

Responsive Design

Dynamic sizing basert på skjermstørrelse

Blir brukt i GameScreen.kt med LocalConfiguration for dynamic heights og font sizes

For å oppfylle krav om 95% device support. Fungerer på 5.5"-7" skjermer.

Feature Oversikt

Feature	Lokasjon	Formål
Jetpack Compose	Alle screens	UI rendering
Navigation	MainActivity	Screen navigation
Room Database	Data/package	Lokal lagring (offline)
Firebase Auth	AuthViewModel, LoginScreen	Google Sign-In
MVVM	Hele appen	Arkitektur-pattern
Coroutines/Flow	ViewModels, DAOs	Async operations
Material 3	Hele appen	Design components
Error Handling	NetworkUtils, LoginScreen	Nettverksfeil
State management	Composables	UI State
Responsive Design	Game Screen	Multi-device support
Logging	Overalt	Debugging

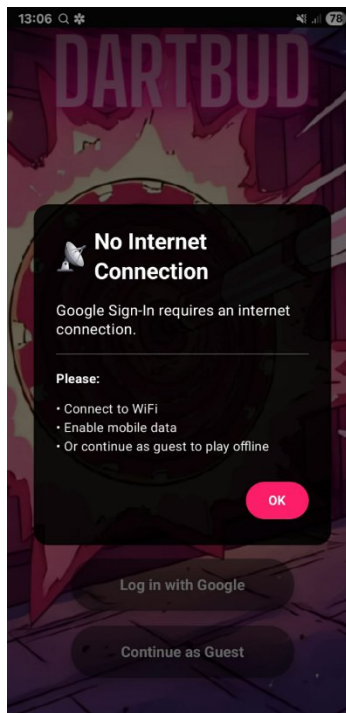
Vi har valgt moderne anbefalte teknologier:

- Jetpack Compose for enklere UI-utvikling
- Room for type-safe lokal database
- Coroutines for enklere async kode
- MVVM for clean architecture
- Material 3 for moderne design

Disse valgene gir oss en moderne, vedlikeholdbar kodebase som følger Android best practices og oppfyller alle kurskrav.

Vi bruker `@Composable` i stedet for fragments, for eksempel ViewModels, `StateFlow/State` som forventet, og senere for bidireksjonell dataflyt i forbindelse med atomisitet i forbindelse med databasehåndtering etter ACID-egenskaper. Vi har brukt `@Composable` der det skal vises ting på skjermen.

Error handling:



Spesifikt for DartBud: Google Sign-In krever internett-tilkobling, får opp feilmelding om internett er borte her. Guest mode skal fungere uten internett. Det kommer opp en AlertDialog med info om hva som må gjøres (koble til internett). Alternativt fortsette som «guest» uten internett.

The database for the application consists of one main table for the users/players, and one table for storing information about games. Additionally, there are tables for tracking current game data and countering many-to-many relationships in the database:

- The `match_player` table joins the player and the match.
- The table joins the value of a single throw from a single player to a round/turn in a game.

Komponenter og biblioteker brukt

Biblioteker:

Vi har brukt en rekke biblioteker som varierer i natur fra kritiske og grunnleggende, til detaljorienterte.

Android.os.Bundle

For å lagre og overføre data, gjennom bruk av key/value-containerer.

Androidx.activity.ComponentActivity

Dette er en basisklasse for alle moderne Android activities, og bruker til å håndtere activity-livssyklus (onCreate, onDestroy, osv.).

Androidx.activity.compose.setContent

Denne er til å "sette" innholdet (content) inn i activityen, for å vise UI. Denne erstatter også XML-layout, slik at man ikke trenger separate filer for layout av «knappene» (innhold).

androidx.activity.enableEdgeToEdge

Dette gjør at appen bruker hele skjermen, inkludert under statusbaren på toppen av skjermen hvor klokke og notifikasjoner vises, samt navigasjonslinjen nederst, og håndterer systembarer automatisk. Uten dette får man hvit bakgrunn under og over selve app-contentet.

androidx.compose.animation.*

Dette er først og fremst en kosmetisk ting, men med animasjoner så kan brukeren også oppfatte lasting som at den tar mindre tid enn det den egentlig gjør. Compose trenger dette biblioteket for myke animasjonsoverganger.

androidx.compose.foundation.layout.padding

UI-elementer må ha luft og avstand for at man skal lese og oppfatte ting lett. Brukes til å holde innhold unna kanter og andre elementer.

androidx.compose.material3.Scaffold

Dette brukes for å gi et standardisert layout for Material Design 3-apper, og gir en konsistent UI-opplevelse for bedre lesbarhet og UX.

androidx.compose.runtime.Composable

Dette trengs for å kalle andre composables og bruke states. Compose trenger også å vite hvilke funksjoner som bygger UI.

androidx.compose.ui.Modifier

Trengs for å kunne justere på utseende og oppførsel på composables. Dette innebærer f.eks padding, bakgrunn, borders, alignments, clickable osv og kan kombineres i kjede for flere effekter. Her er rekkefølgen også viktig da padding før background gir en annen effekt enn motsatt. Modifier er en grunnleggende byggekloss for all Compose-tilpasning

androidx.lifecycle.viewmodel.compose.viewModel

Dette brukes for å holde staten ved rekomposisjon og eventuelle konfigurasjonsendringer som for eksempel skjermrotasjon. Uten dette biblioteket hadde man simpelthen mistet alle states hver gang skjermen roteres eller recomposer.

androidx.navigation.NavType

Dette trengs for å sikre dataintegriteten, spesifikt datatype mellom skjermbilder. Dette biblioteket definerer type for navigation-argumenter, og sikrer type-sikkerhet ved data transfer mellom destinations.

com.group1.dartbud.ui.theme.DartBudTheme

Dette er for å sette et eget preg på appen, i stedet for standard Android native tema. Siden det er en type «spill-app» vil vi ha litt livlige farger og knapper tilpasset store pølsefinger.

androidx.navigation.compose.NavHost

NavHost er en samlecontainer som holder alle navigation destinations, og viser riktig composable basert på nåværende route. Den integrerer også native med Androids tilbake-knapp. Det er en grunnleggende navigasjonsstruktur for multi-screen Compose-apper.

androidx.navigation.compose.composable

Hver composable er én skjerm i appen, og brukes av NavHost for å vite hvilke skjermer som finnes og hvordan de bygges. Composable definerer en navigation destination, kobler route-string til composable-funksjoner, tar imot argumenter fra navigasjon og bytter skjermbilde når brukeren navigerer dit. Uten dette har NavHost ingen destinations å navigere til.

androidx.navigation.compose.rememberNavController

Oppretter og husker NavController-instant, holder styr på destinasjoner, og sendes til NavHost og andre composables som trenger navigasjon. Uten remember ville man fått ny NavController ved hver rekomposisjon og mistet navigasjonshistorikk.

androidx.navigation.navArgument

Definerer hvilke argumenter som Navigation routes aksepterer og hvordan. Brukes til å spesifisere navn, type og egenskaper for route-argumenter, og brukes i composable()'s arguments-liste. Uten dette får man runtime-feil ved feil datatyper. Gir type-safety.

Androidx.compose.foundation.layout.* (Row, Column, Box, Spacer, fillMaxWidth, fillMaxSize, size, width, height, etc.)

Brukes for å definere hvordan UI-elementer arrangeres og plasseres på skjermen, mye som .xml-layout filene man brukte i gamle dager.

Androidx.compose.foundation.shape.RoundedCornerShape

Form i Jetpack Compose som brukes til å lagre rektangler med avrundede hjørner.

Brukes som regel sammen med Modifier-funksjoner.

Android.compose.foundation.rememberScrollState

Brukes for å opprette og huske et ScrollState-objekt. Reagere på rullehendelser.

Androidx.compose.foundation.verticalScroll

Modifier som gjør at en Composable kan rulles vertikal. Brukes typisk sammen med `rememberScrollState()` for å kontrollere og observere rullestatusen. Egnert for innhold som er for høyt for skjermen men ikke så omfattende at man krever ytelsesoptimaliseringen til `LazyColumn`. Vi har brukt denne til et tekstfelt som forklarer reglene for 501 Dart.

Androidx.compose.foundation.Image

Composable for å vise bilder i JetPack Compose. Støtter formater som `ImageVector`, `Bitmap`, `Painter` og `ImageBitmap` og lastes typisk fra ressurser med `painterResource()`. Tilpasses ofte med Modifier. Vi bruker dette for å vise et bilde med DartBud-logo og aktiv dartspilling, som modifies til brukerens skjermstørrelse.

Androidx.compose.material.icons.* (Icons, automirrored, filled)

Gir tilgang til Material Design-ikoner i forskjellige stiler. Brukes med `Icon`-komponenten for å vise vektografikk-ikoner i UI-et. `AutoMirrored`-varianter sikrer automatisk speilvending av ikoner i RTL-locale.

Androidx.compose.material3.* (`Button`, `OutlinedButton`, `Icon`, `Text`, `TopAppBar`, `Card`, etc.) Implementering av Material Design 3-komponenter for å skape konsistente og moderne UI. Inkluderer `Button`, `OutlinedButton`, `Icon`, `Text`, `TopAppBar`, `Card`.

Androidx.compose.runtime.* (`remember`, `mutableStateOf`, `rememberCoroutineScope`, etc.) Dette er kjernen i Jetpack Compose. Brukes til å håndtere tilstand, komposisjon og lifecycle i UI. Nøkkelfunksjoner innebærer `@Composable`-annotasjon, `remember` (lagring av verdier for å lagre state mellom rekonponeringer), `mutableStateOf` (oppretting av observerbar tilstand som utløser rekonponering ved endring (f.eks `scroll`)), `LaunchedEffect` (asynkrone operasjoner), `DisposableEffect` (håndtering av ressursintensive operasjoner som krever opprydding ved avslutting) osv.

Androidx.compose.ui.Alignment

Enum-klasse for å oppnå ønsket layout og hierarki i Compose-baserte UI. Brukes til å definere plasseringen av UI-elementer i en parent container, og spesifiserer hvordan children justeres relativt til forelderen og er avgjørende for presis kontroll over layout.

Androidx.compose.ui.geometry.* (`CornerRadius`, `Offset`)

Brukes for å avrunde kanter på elementer. `Offset` brukes av og på i forbindelse med en pil som viser hvilken spiller sin tur det er til å kaste dartpiler.

Androidx.compose.ui.graphics.* (`Color`, `Shadow`, `drawscope.Stroke`)

Brukes for å legge farge, skygge og kontur på elementer.

Androidx.compose.ui.layout.ContentScale

Brukes til å definere hvordan innhold, spesielt bilder, skal skaleres og beskjæres for å passe inn i containere i Jetpack Compose. Avgjørende for å bestemme hvordan visuelle elementer, som bilder, skal vises, uten å forvrengte proporsjonene. Vanlige verdier inkluderer `ContentScale.Crop`, `ContentScale.Fit`, `ContentScale.FillBounds`, `ContentScale.Inside`.

Androidx.compose.ui.platform.LocalConfiguration

Gir tilgang til enhetens konfigurasjonsobjekt (Skjermorientering, språk, skriftstørrelse). Brukes for å tilpasse UI dynamisk, basert på enhetsmiljøet uten å eksplisitt overføre data gjennom komposisjonstrær. For vår del gjelder dette håndtering av ulike skjermstørrelser for responsiv design.

Androidx.compose.ui.platform.LocalContext

Gir tilgang til Androids Context-objekt som er nødvendig for alle operasjoner som krever systemressurser eller interaksjon med Android-frameworket. Dette gjelder henting av ressurser, systemtjenester, filhåndtering, tilgangsstyring og nettverksstatus. CompositionLocal som LocalContext bygger på er designet for å unngå prop drilling av kontekstuelle data gjennom komposisjonstrær som gjør koden mer vedlikeholdbar og lesbar.

Androidx.compose.ui.res.painterResource

Hjelpfunksjon som laster inn bilder fra f.eks drawable-mappen og konverteres til et Painter-objekt. Henter bilder fra appens ressurser, men viser det ikke direkte (gjøres av Image-biblioteket). Avgjørende for integrering av statisk bildegjenstand fra appens ressurser uten eksterne biblioteker. painterResource er en som en bildeleser som henter filen fra lagring, mens Image er skjermen som viser bildet. De må brukes sammen.

Androidx.compose.ui.text.* (TextStyle, font.FontWeight, style.TextAlign)

Bibliotek som håndterer tekstformatering og stil. Kombineres typisk med TextStyle, FontWeight og TextAlign i en Text-komponent for presis kontroll over tekstutseende.

Androidx.compose.ui.unit.dp

Brukes for å skalere elementer til skjerm i stedet for å bruke faste størrelser, da skjermer kan ha lik oppløsning, men være både 5 tommer og 42 tommer. Ved et enkelt mattestykke

Androidx.compose.ui.unit.sp

Konverterer numeriske verdier til TextUnit-objekter med scalable pixels (sp) som enhet. Dette sikrer at tekst automatisk skalerer basert på brukerens systempreferanser for tekststørrelse, noe som er kritisk for å støtte brukere med synshemming. Dette er en Android best practice for tilgjengelighet og JetPack Compose forventer sp for alle tekstrelaterte størrelser.

Com.group1.dartbud.utils.NetworkUtils

Custom utility-klasse for nettverkssjekk. Brukes til å detektere internett-tilkobling før nettverksavhengige operasjoner som Google Sign-In. Implementert for å oppfylle krav om error handling. Returnerer boolean basert på enhetens nettverksstatus via ConnectivityManager

Android.net.ConnectivityManager / NetworkCapabilities

System-tjeneste for å sjekke nettverkstilstand. Brukes i NetworkUtils for å sjekke at enheten har internett-tilkobling som fungerer.

Kotlinx.coroutines.launch

For å starte en ko-rutine, eller en «kodeblokk som kan pause og fortsette» uten å blokkere hovedtråden (UI-tråden). Brukes til lengevarende operasjoner uten fryse

appens grensesnitt (lagring av poensum til database, henting av spillerprofil fra internett)

Java.text.SimpleDateFormat

Brukes til å formatere og analysere datoer og klokkeslett

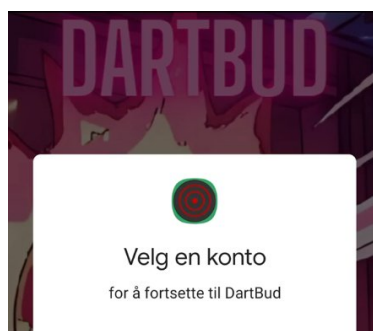
End Product

Installering og kjøreinstruksjoner

Gå til <https://github.com/JonasELH/DartBud> og last ned .apk-filen «app-debug.apk» fra «releases». Siden den installeres fra utenfor Play Store vil du få et dialogvindu opp hvor det spørres om du stoler på kilden. Trykk ja på denne for å få installert appen, eller nei om du ikke stoler på kilden. Da får du heller ikke installert appen. Alternativt, bruk [direktelenken](#).

Du kan også (om vi rekker det), gå på Googles offisielle Play Store, søk på «DartBud» og last ned appen derfra.

App-funksjonalitet



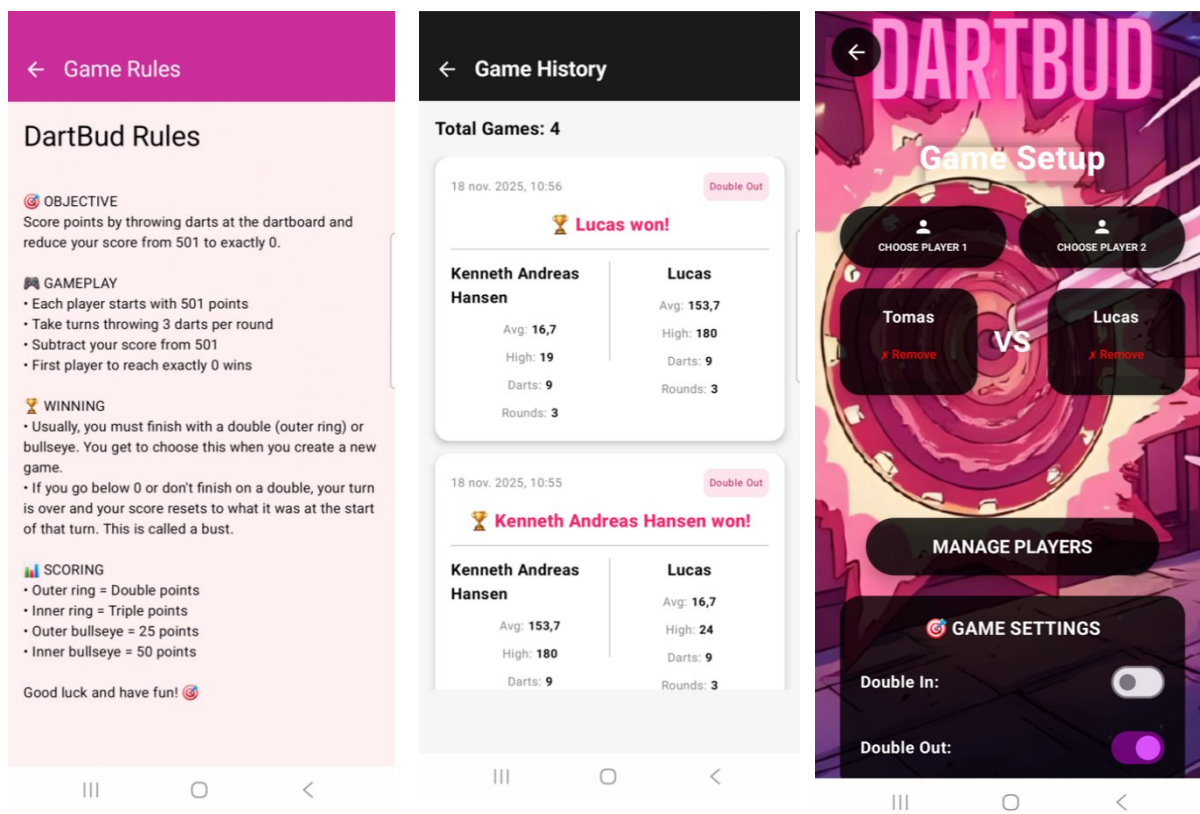
Appen vår har full funksjonalitet for å spille en 501-dartkamp, lokalt. Vi gir muligheten for å spille som gjestebruker ved å trykke på «Continue as guest» knappen når man først åpner appen. I tillegg har vi implementert innlogging med Google-konto som vil tillate at spillhistorikk og annen nyttig statistikk lagres for de som er ekstra interesserte.

Pr nå så har vi valgt å ha det slik at spillehistorikk ikke lagres for spillere som velger å spille lokalt. Dette er mer for vår egen del da bak tanken med dette er at vi ønsker at brukere skal ha et insentiv for å oppgi personal informasjon, det er en kjent fordel at alle «virksomheter» trenger slik info til forretningsanalyse formål.

..

De som eventuelt er personvernsbevisste rundt hvilken data de gir ut for også muligheten til å spille fullstendig «offline» og slipper å oppgi persondata.

De som definerer seg mer som «Casual brukere» trenger som oftest likevel ikke detaljert statistikk og «ekstra funksjonalitet», utover å kunne spille en vanlig kamp og lukke appen når de er ferdig igjen.



For nybegynnere finnes en “Rules”-knapp i startmenyen som gir en enkel oversikt over reglene som brukes i spillet. Disse er formulert på en måte som gjør at folk som aldri har spilt dart før skal kunne sette i gang på egen hånd.

Vi har også en enkel «Game history» side som viser hvem som vant tidligere kamper med enkel statistikk som viser hvordan deltakerene spilte ...

Når man ønsker å starte en kamp, får man opp alternativer for å velge spiller 1 og spiller 2. Her kan man enten legge til nye spillere eller velge blant eksisterende brukere, dersom de allerede er opprettet på enheten.

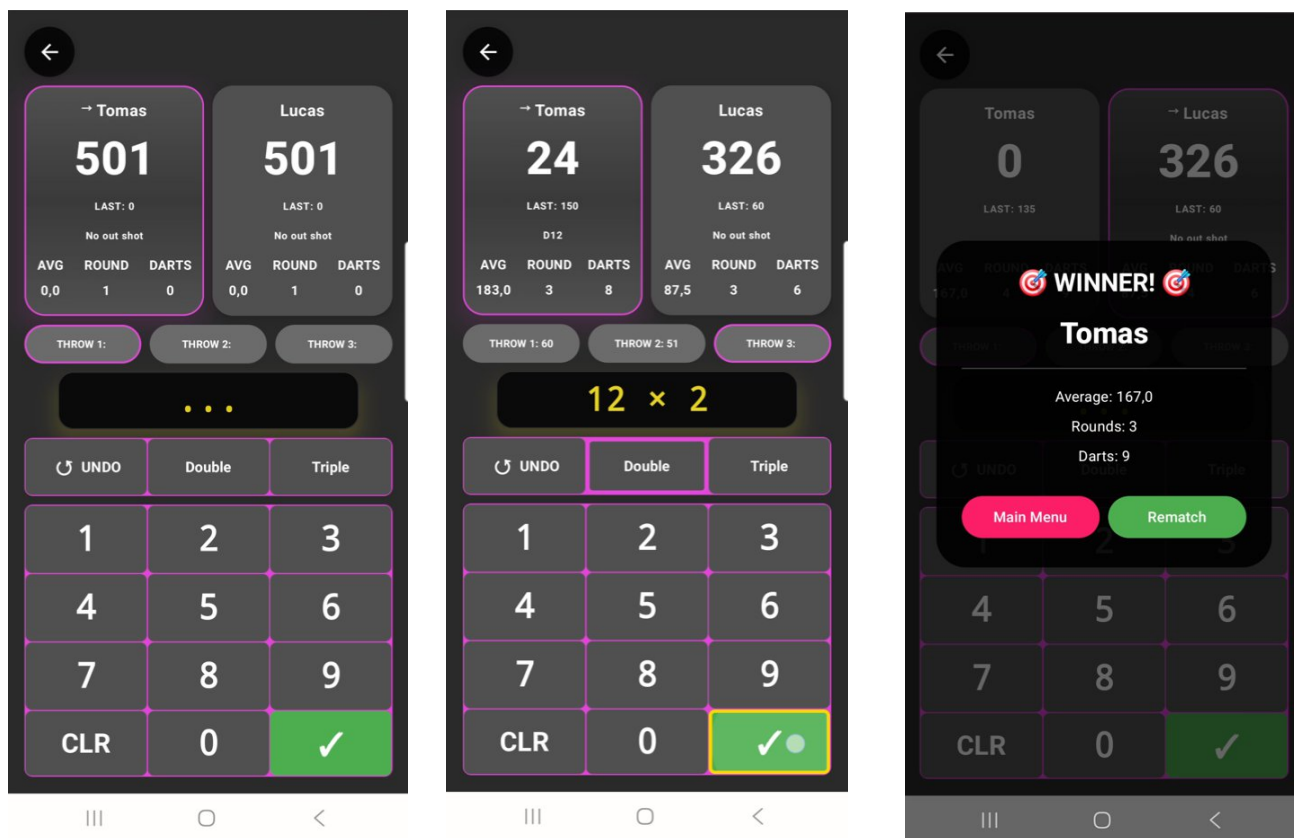
Før kampen starter kan man velge om man vil spille med “Double In”, “Double Out”, eller ingen av delene.

Dette påvirker hvordan spillet starter og avsluttes:

- **Double In:** Man må treffe et rødt eller grønt felt i ytterkanten av dartsdividen før man kan begynne å samle poeng.
- **Double Out:** Man må avslutte spillet med et treff i et slikt felt for å kunne vinne runden.

Når oppsettet er klart og kampen starter, vises en skjerm som ligner en kalkulator. Her kan man enkelt legge inn poengene for hvert kast. Skjermen gir også oversikt over spillernes poengsummer, hvem sin tur det er, og hvor mange kast hver spiller har brukt.

Underveis vises enkel statistikk, som for eksempel gjennomsnittlig poengsum per runde. Dersom man treffer dobbel- eller trippelfelt, kan man trykke på “Double” eller “Triple”-knappen før man legger inn poengsummen, beregningen vil utføres automatisk i appen.



Når man starter en kamp så vil man få en intuitiv og oversiktlig spillerskjerm. Her kan man enkelt legge inn poengene for hvert kast. Skjermen viser hvem sin tur det er, gir en oversikt over spillernes poengsummer og hvor mange kast hver spiller har brukt. Underveis vises enkel statistikk, som for eksempel gjennomsnittlig poengsum per runde. Dersom man treffer dobbel- eller trippelfelt, kan man trykke på “Double” eller “Triple”-knappen før man legger inn poengsummen, beregningen vil utføres automatisk i appen. Har man trykket inn feil poengsum så kan man på enkelt vis trykke på «Undo» knappen og man vil få muligheten til å skrive inn poengsummen på nytt.

Hvis man har skrudd på spillfunksjonen for «Double in» eller «Double out» så må man treffe krevd poengsum og trykke på «double» knappen for å starte eller avslutte kampen. Hvis man ikke gjør dette, så vil man ikke komme videre og å trykke på den grønne knappen vil ikke fungere. Dette er implementert for å gjøre det tydelig for spillere at de faktisk må treffe en double for å gå videre.

Etter en kamp er ferdig vil man få opp en knapp for å gå tilbake til «Main menu» hvis man ønsker å avslutte eller «Rematch» hvis man ønsker å spille en ny runde mot samme motstander.

Brukermanual (valgfritt/kort)

Denne ligger i separat vedlagt Word-dokument!

Testing

Testing av appen

Testing: Samsung Galaxy S22 og S22 Ultra (fysiske enheter) - Android Emulator: Medium Phone API 34, Pixel 9a

Vi har testet alle kjernefunksjoner i appen gjennomgående parallelt med utvikling:

Error handling: Skrudde av internett før jeg prøvde å logge inn med googlekonto, fikk opp feilmelding/alertdialog med hva som måtte gjøres.

501 game logic: Score countdown, bust detection, winner detection - Double/Triple calculations:

Verifisert korrekt matematikk - Double In/Out regler: Testet at regler fungerer som forventet

Statistikk: average, high score, darts thrown, alle fungerer som forventet

Undo-system: Testet undo av individuelle kast og på tvers av spillere.

Checkout suggestions: Verifisert korrekte forslag for 2-170 poeng.

Database: Google Sign-In, local storage, game history lagring.

UI/UX TESTING: - Responsive design: Testet på ulike skjermstørrelser (6.4 (Medium Phone), 6.24 (Pixel 9a), 10.95 (Pixel Tablet) tommer, og det fungerte. På mindre skjermer kan man scrolle i menyene, og på hovedskjermen tilpasser widgetene seg til viewporten. I liggende modus blir tallene borte på mindre skjermer, men det fungerer fint på nettbrett.

Dark mode: Fikset tekstsynlighet i både light og dark mode, etter at dark mode viste svart tekst på svart Button.

Navigation: Verifisert flyt mellom alle screens og back button.

REAL-WORLD TESTING: Kenneth testet appen hjemme mot en kompis fra dartklubben "Viul Pilkastarlaug" i Hønefoss. Setup var standard dartbrett med fysiske piler, full 501-kamp med double out aktivert på Samsung S22. Resultat: Appen fungerte feilfritt gjennom hele kampen. Ingen kræsjer, undo-funksjonen fungerte som forventet, og checkout forslag var nyttige.

Konklusjon: Testing viser at appen er stabil og produksjonsklar. Den krasjer ikke tilfeldig. Alle kritiske bugs er fikset. Real-world testing viser at appen løser behovene vi hadde forespeilet oss. Man kan bruke den for å løse problemområdet med hoderegning, og skal man «jukse» må man gå inn for det med overlegg.

Evaluation Conclusion

Hva er ferdig/ikke ferdig

Uferdig:

Online multiplayer i sanntid.

Webkamera løsning

Muligheter for andre fargetema/skins i appen

Ferdig:

Error handling ved loginskjerm/Google-innlogging

Offline (guest mode fungerer uten nett) uten lagring av data (ref dataminimering GDPR)

Fremtidige forbedringer

Mulige påtenkte forbedringer til senere utvikling kan for eksempel være implementering av webkamera. At man kan sette opp mobilen slik at man kan filme samtidig som man spiller fra hver sin telefon. Andre forbedringer kan være at man har muligheten for en chatfunksjon i et vindu. Online Multiplayer: En mulighet ville være å la spillere konkurrere mot hverandre på tvers av enheter via internett. Dette ville kreve: - Firebase Realtime Database for game state synkronisering, Venneliste/matchmaking system - Tur-basert spilllogikk med disconnect-håndtering, push notifications for invitasjoner. Dette ble nedprioritert til fordel for solid lokal multiplayer-funksjonalitet og omfattende testing. Ellers kan alltid UI bli bedre, med tanke på fargekoder, brukergrensesnitt, for eksempel å bestemme knappefarger og outline-farge. Men vi har jobbet etter beste evner her og vår forståelse av design.

Guest-brukere får basic spill historikk (vinner, regler, tidspunkt) lagret lokalt, mens innloggede brukere får i tillegg full statistikk (average, high score, darts thrown). Dette er et bevisst designvalg for å oppmuntre registrering og samtidig tenke på GDPR.

Individuell refleksjon

Individuell refleksjon	
Jonas	Jeg er generelt fornøyd med prosjektarbeidet vårt. Det har vært veldig interessant å se hvordan mange av de samme systemutvikling og programmerings prinsippene kan benyttes universelt uansett og man lager en web applikasjon som vi har fått litt erfaring med eller om det er

	en mobil app. Jeg synes vi har jobbet relativt effektivt med tanke på at man også må lære nye programmeringsspråk og IDE, samtidig som man er ment til å lage et reelt produkt. Vi har jobbet jevnt over semesteret som jeg føler har vært til stor fordel. Videre føler jeg at vi har vært flinke til å kommunisere mellom oss om arbeidsoppgaver, forvetninger og hvordan vi skulle fordele det som måtte gjøres. Det er selvfølgelig alltid forbedringspotensielle når man ser tilbake i etterkant, men alt sett i sammenheng så er jeg stolt av innsatsen vår.
Lucas	I begynnelsen var det litt tungt å få applikasjonen til å fungere for å testes på alle medlemmers pc på grunn av dependecies og ulike errormeldinger. Så snart vi fikk ryddet opp i det, gikk prosessen fortere fremover. Noe annet som skapte problemer, har vært bruken av Github hvor jeg personlig enda ikke har klart å bygge en intuitiv forståelse for seamless bruk. Jeg deltok mest i begynnelsen av prosjektet, hvor vi fikk en nogen lunde brukbar GUI på plass, og jobbet med det grunnleggende av funksjonalitet i henhold til usecases osv. men føler at jeg har sklidd ut siste perioden på grunn av flytting og påbegynnelse av et ekstra studie. Alt i alt har gruppa som vanlig vært målrettet og konsistente med arbeidet rundt prosjektet. Ettersom vi allerede har vært en gruppe en god stund fra før, har dette vært et prosjekt hvor medlemmene tidlig inntok "roller" etter hva man ønsker/føler seg best i å jobbe med. Vi har også vært flinke til å ta de mest utfordrende sakene i plenum. Kombinasjonen gjør at effektiviteten øker drastisk.
Kenneth	Det jeg savner er hovedsakelig en god prosess rundt Github og hvordan jobbe med det på en god måte. Skulle gjerne ønske meg at vi hadde en uke eller to hvor vi jobbet med dette på skolen for å forstå hvordan man jobber i det profesjonelt. Slik det er nå er det bare en learning by doing metode og for meg så har ikke det fungert så bra. Føler det har blitt merarbeid på grunn av det. Under utviklingen kommer man på stadige mulige forbedringer, men kompetansen er ikke helt der oppe med hva man forespeiler seg i tankene sine. Skulle gjerne gitt ut på PlayStore men er litt redd for "reglene" m.t.p hva man må ta hensyn til i forhold til datasikkerhet, personvern og hva man må få bruker av systemet til å huke av på/godkjenne og den biten der. Livredd for søksmål for at man har en sikkerhetsfeil i appen som gjør at persondata kommer på avveie og man blir fremstilt som ansvarlig. Ellers har hele prosessen vært relativt knirkefritt, noen intense økter, andre rolige. Ingen kjempestore feilmeldinger vi aldri fikk løst. Det vanskeligste var kanskje å begynne, men da ballen først var begynt å rulle så gikk det fremover.
Kristian	Det har vært lærerikt å bli kjent med prosessen med applikasjonsutvikling for Android, og hvordan det henger sammen. Det var overraskende at emulatoren var en hel bildefil av et Android-system, komplett med AT&T-radio som default mobilnett. Videre ser man igjen kjente konsepter fra andre kodedomener som lifecycles i React og Svelte, som man også finner igjen i Kotlin. Vi hadde nok kunne hatt en raskere utvikling dersom vi brukte git branches på korrekt vis og vi hadde noen på laget som kunne vite mer sikkert hvilke commits man skulle velge å beholde når det blir merge conflicts. Nå

	«pusher vi til main» hele tiden. Videre så var det også greit at vi hadde tilgang til flere ulike fysiske mobiler da selve hovedskjermen for darkkalkulator der man taster inn score-inputen vises likt på den virtuelle Medium Phone og på S22 Ultra, og annerledes på S22 på grunn av fysisk skjermstørrelse. Vi ville unngå konstant scrolling på selve hovedskjermen, trade-offen er at de ikke fungerer så bra på mindre mobiler i liggende modus. Dette viser viktigheten av valgene man gjør på koden og betydningen av piksel tetthet, tetthetspunkter (dp), px, % osv og at spesifisiteten på dette må tilpasses til brukersegmentet. Hadde denne appen vært tiltenkt på budsjett-telefoner (typ de man installerte Firefox OS på) måtte vi nok også ha justert mengden informasjon som vises på skjermen, spesielt i P1+P2-kortene på hovedspillarskjermen. Har tidligere tenkt at det var teit og «extra» å ha egne stativer for å holde mobilenheter, men etter å ha låst opp telefonen en del ganger i forbindelse med Android Studio ser jeg nå nytten.
--	---

9. AI-bruk:

Anthropic. (2024). Claude AI. Hentet fra <https://claude.ai> Brukt til: Debugging, kodeforslag, hjelp til forståelse av komponenter, problemforståelse, debugging av bust detection logikk, forståelse av designkonsepter, hjelp til Room database relasjonshåndtering, Jetpack Compose syntaks og eksempler

OpenAI. (2024). ChatGPT - AI Language Model. Hentet fra <https://chat.openai.com> Brukt til: Kodeforslag, problemløsning, syntaks-hjelp og konseptuell forståelse av Android-komponenter.

10. References

Datatilsynet. (2019). *Personvernprinsippene*. Hentet fra <https://www.datatilsynet.no/rettigheter-og-plikter/personvernprinsippene/grunnleggende-personvernprinsipper/dataminimering/>